

Question Bank — Week 2

Complexity and the Array · control flow, functions, errors (Lecture 02, Lab 02) —
Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Answers are in a separate file: [week02-answers.md](#). Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify. Unless a question says otherwise, n is the input size and `Array` is the course array from `dsa/array.py`.

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W2-M01 – W2-M26
B	Short answer and essay	W2-E1 – W2-E6
C	Complexity analysis — give Θ and justify	W2-K1 – W2-K6
D	Proofs	W2-P1 – W2-P3
E	Array state — show every step	W2-S1 – W2-S4
F	Trace the code	W2-T1 – W2-T4
G	Find and fix the bug	W2-B1 – W2-B3
H	Write the code — checked by pytest	W2-C1 – W2-C4

Part A — Multiple choice

W2-M01 [how] $f(n) = 3n^2 + 5n + 2$ is:

- **a)** $\Theta(n)$
- **b)** $\Theta(n^2)$
- **c)** $\Theta(n^3)$
- **d)** $\Theta(n \log n)$

W2-M02 [why] Which statement is **true**?

- **a)** $n^2 = O(n)$
- **b)** $n = \Omega(n^2)$
- **c)** $n = O(n^2)$
- **d)** $2^n = O(n^3)$

W2-M03 [what] $f(n) = O(g(n))$ means:

- **a)** there are constants $c > 0$ and n_0 with $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

- **b)** $f(n) \leq g(n)$ for every n
- **c)** $f(n) = g(n)$ for all large n
- **d)** there are constants $c > 0$ and n_0 with $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$

W2-M04 [how] In the **worst case**, how many comparisons does linear search make on an array of n elements?

- **a)** 1
- **b)** $n / 2$
- **c)** $\log_2 n$
- **d)** n

W2-M05 [how] The target is present and equally likely to be at any of the n positions. On **average**, how many comparisons does linear search make?

- **a)** $(n + 1) / 2$
- **b)** n
- **c)** $\log_2 n$
- **d)** $n^2 / 2$

W2-M06 [how] What is the running time of this loop?

```
i = n
while i > 1:
    i = i // 2
```

- **a)** $\Theta(1)$
- **b)** $\Theta(n)$
- **c)** $\Theta(\log n)$
- **d)** $\Theta(n/2)$

W2-M07 [how] What is the running time?

```
for i in range(n):
    for j in range(i):
        count += 1
```

- **a)** $\Theta(n)$
- **b)** $\Theta(n \log n)$
- **c)** $\Theta(i \cdot n)$
- **d)** $\Theta(n^2)$

W2-M08 [how] What is the running time?

```
for i in range(n):
    for j in range(5):
        count += 1
```

- **a)** $\Theta(n)$
- **b)** $\Theta(n^2)$
- **c)** $\Theta(5^n)$
- **d)** $\Theta(\log n)$

W2-M09 [what] For a Python list of n items, `x in values` costs:

- **a)** $O(1)$, because lists are hash tables
- **b)** $O(n)$
- **c)** $O(\log n)$
- **d)** $O(n^2)$

W2-M10 [what] Which Python list operation is $O(1)$ (amortised)?

- **a)** `values.insert(0, x)`
- **b)** `values.pop(0)`
- **c)** `values.append(x)`
- **d)** `values.remove(x)`

W2-M11 [why] Why is reading `a[i]` from an array $O(1)$?

- **a)** The array is sorted
- **b)** Python caches the last element read
- **c)** The array searches from both ends at once
- **d)** The slots are contiguous and equal-sized, so the address is $\text{base} + i \times \text{slot size}$

W2-M12 [how] A partly filled array has **size 6**. How many elements must move to insert a new value at **index 2**?

- **a)** 4
- **b)** 2
- **c)** 6
- **d)** 3

W2-M13 [how] When inserting into an array, the elements after the gap must be shifted right. In which order?

- **a)** From the insertion index towards the end
- **b)** From the last used slot back towards the insertion index
- **c)** Any order — the result is the same
- **d)** Swap each element with its neighbour, from the front

W2-M14 [how] `a = Array(4)`, then `a[-1]`. What happens?

- **a)** returns the last slot, `None`
- **b)** returns `a[3]`
- **c)** raises `IndexError`
- **d)** raises `TypeError`

W2-M15 [how] `a = Array(5)`, then `a[0] = 7`. What is `len(a)`?

- **a)** 1
- **b)** 0
- **c)** 6
- **d)** 5

W2-M16 [how] A 3×4 matrix (3 rows, 4 columns) is stored flat in row-major order. At which index is element (2, 1)?

- **a)** 7
- **b)** 9
- **c)** 6
- **d)** 11

W2-M17 [how] What extra space does `reverse_in_place(arr)` need (swap from both ends, no second array)?

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n^2)$

W2-M18 [why] Under the course storage rule (Lecture 02), which of these is **not** allowed?

- **a)** a Stack storing its items in the student's own `DynamicArray`
- **b)** a Graph storing adjacency in the student's own `ChainingHashMap`
- **c)** a queue class storing its items in a Python `list`
- **d)** bfs returning the visit order as a Python `list`

W2-M19 [what] Python's built-in `list` is best described as:

- **a)** a linked list of values
- **b)** a fixed-size C array of integers
- **c)** a hash table keyed by position
- **d)** a dynamic array of references to objects

W2-M20 [how] An $O(n)$ loop is followed by an $O(n^2)$ loop. The whole is:

- **a)** $O(n^2)$
- **b)** $O(n^3)$
- **c)** $O(n)$
- **d)** $O(n + n)$

W2-M21 [how] `values` is a Python list of n items. The cost of:

```
for i in range(len(values)):
    if values[i] in values[i + 1:]:
        return True
```

- **a)** $O(n)$
- **b)** $O(n^2)$
- **c)** $O(n \log n)$
- **d)** $O(1)$

W2-M22 [what] When does the `else` block of a `for` loop run?

- **a)** Whenever the loop body raises an exception
- **b)** Only if the loop runs zero times
- **c)** When the loop finishes without executing `break`
- **d)** After every iteration

W2-M23 [how] Given

```
def f(a, L=[]):  
    L.append(a)  
    return L
```

what does `f(1)` then `print(f(2))` print?

- a) [2]
- b) [1]
- c) [2, 1]
- d) [1, 2]

W2-M24 [how] What is the value of `0` or `'x'` and `''`?

- a) `''`
- b) `'x'`
- c) `0`
- d) `False`

W2-M25 [how] `list(range(2, 11, 3))` is:

- a) [2, 5, 8, 11]
- b) [2, 5, 8]
- c) [3, 6, 9]
- d) [2, 3, 4, 5, 6, 7, 8, 9, 10]

W2-M26 [how] What does `print(g())` print?

```
def g():  
    try:  
        return 1  
    finally:  
        print("bye")
```

- a) 1
 - b) bye
 - c) bye then 1
 - d) 1 then bye
-

Part B — Short answer and essay

W2-E1 [why] (4 marks) Big-O throws away constant factors and lower-order terms. Explain *why* each is thrown away, give one situation where doing so is misleading, and explain how measuring complements counting.

W2-E2 [what] (4 marks) Define O , Ω and Θ precisely. For $f(n) = 3n^2 + 2n$, give one true statement using each, and one *true but uninformative* O statement.

W2-E3 [why] (3 marks) Describe the best, worst and average case of linear search, with the number of comparisons for each. Which one does "linear search is $O(n)$ " usually refer to, and why is

that the default?

W2-E4 [why] (4 marks) Explain why indexing an array is $O(1)$ but inserting into the middle is $O(n)$. What does the fact that an array has a *fixed size* force you to do when it fills up, and what does that cost?

W2-E5 [why] (4 marks) State the course storage rule from Lecture 02. Give the reason for it, list what remains allowed, and show how two structures from later in the course follow it.

W2-E6 [what] (3 marks) Compare Python's `list`, `array.array`, `numpy.ndarray` and a `ctypes` array on: can it grow, what it holds, and whether it hides costs. Why is the course `Array` built on `ctypes`?

Part C — Complexity analysis

Give the running time in Θ notation as a function of n , and **justify** it in one or two sentences (which rule, which sum). Where best and worst differ, give both.

W2-K1 [how]

```
total = 0
for i in range(n):
    for j in range(n):
        total += i * j
for k in range(n):
    total += k
```

W2-K2 [how]

```
i = 1
while i < n:
    i = i * 2
```

W2-K3 [how]

```
for i in range(n):
    j = n
    while j > 1:
        j = j // 2
```

W2-K4 [how] (`arr` is an `Array` of n elements.)

```
def has_duplicate(arr):
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] == arr[j]:
                return True
    return False
```

W2-K5 [how] (`values` is a Python list of n items.)

```
for i in range(n):  
    rest = values[i:]
```

W2-K6 [how]

```
count = 0  
for i in range(n):  
    for j in range(100):  
        for k in range(i):  
            count += 1
```

Part D — Proofs

W2-P1 [how] (3 marks) Prove that $2n + 10 = O(n)$ by giving c and n_0 and showing the inequality.

W2-P2 [why] (3 marks) Prove that n^2 is **not** $O(n)$.

W2-P3 [how] (4 marks) Prove that $4n^2 + n = \Theta(n^2)$.

Part E — Array state

Draw the array after each step (use `_` for an unused slot) and count the elements moved.

W2-S1 [how] An Array of capacity 7 holds `[10, 20, 30, 40, 50, _, _]`, size 5.

- (a) `size = insert_at(arr, size, 1, 15)`
- (b) then `remove_at(arr, size, 3)` (and size decreases by one)

W2-S2 [why] A student writes the shifting loop in insertion **forwards**:

```
for i in range(index, size):  
    arr[i + 1] = arr[i]  
arr[index] = value
```

Show the array after inserting "X" at index 1 into `["a", "b", "c", "d", _]`, size 4. What went wrong, and what is the fix?

W2-S3 [how] `rotate_left` by three reversals. Show the array after each reversal when rotating `[1, 2, 3, 4, 5, 6]` left by 2.

W2-S4 [how] The 3×4 matrix holding the values 0 to 11 row by row is stored flat in row-major order.

- (a) Which value is at (1, 3)? At which flat index?
- (b) Which (row, column) is flat index 10?

- (c) After `transpose_flat` (now 4×3 , row-major), at which flat index is the value that was at (1, 3)?
-

Part F — Trace the code

Write exactly what is printed.

W2-T1 [how]

```
for n in range(10, 16):
    for d in range(2, n):
        if n % d == 0:
            break
    else:
        print(n, end=" ")
print("done")
```

W2-T2 [how]

```
def run(command):
    match command.split():
        case ["add", a, b]:
            return int(a) + int(b)
        case ["neg", a]:
            return -int(a)
        case ["sum", *rest]:
            return len(rest)
        case _:
            return "?"

print(run("add 2 3"), run("neg 4"), run("sum"), run("sum 1 2 3"), run("add 1"))
```

W2-T3 [how]

```
def f(x):
    try:
        print("A", end=" ")
        y = 10 // x
        print("B", end=" ")
    except ZeroDivisionError:
        print("C", end=" ")
        return 0
    else:
        print("D", end=" ")
        return y
    finally:
```



```
        print("E", end=" ")

print(f(2))
print(f(0))
```

W2-T4 [how]

```
def change(values, n):
    values.append(n)
    n = n * 10
    values = [0]
    return n

data, k = [1], 2
result = change(data, k)
print(data, k, result)
```

Part G — Find and fix the bug

W2-B1 [how] Should return True exactly when n is prime.

```
def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5)):
        if n % d == 0:
            return False
    return True
```

W2-B2 [how] Should return the index of target in arr, or -1.

```
def find(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    else:
        return -1
```

W2-B3 [how] Should remove and return arr[index] from a partly filled array, shifting the rest left.

```
def remove_at(arr, size, index):
    value = arr[index]
    for i in range(index, size):
        arr[i] = arr[i + 1]
    return value
```

Part H — Write the code

In `practice/week02.py`; check with `pytest tests/test_practice_week02.py -v`. Work on the Array with indices only, and meet the stated complexity.

W2-C1 [how] `count_occurrences(arr, target, size=None)` — $O(n)$.

W2-C2 [why] `remove_all(arr, size, target)` — remove every occurrence in **one pass**, $O(n)$ time and $O(1)$ extra space; return the new size. Why is calling `remove_at` once per occurrence $O(n^2)$?

W2-C3 [why] `two_sum_sorted(arr, target)` — in a **sorted** array, find $i < j$ with `arr[i] + arr[j] == target` in $O(n)$. Why is it correct to move only one index at each step?

W2-C4 [how] `prefix_sums(arr)` — a new Array with running totals, $O(n)$. How does it let you answer “sum of `arr[i..j]`” in $O(1)$?